Narration Studio as a full AI agentic system

The **option** of evolving this app from a scripted multi-agent factory into a full AI agentic system. This is not how the app works today.

> **Confirmed** means current behavior in this repository's source. **Proposed** means work that is not implemented. **Inferred** means industry definition, not a type in this repo. Related: `docs/harness-engineering.md` (runtime scaffold). A harness is necessary but not sufficient for an agentic product.

Verdict

Narration Studio is a **scripted multi-agent pipeline** with a deterministic fallback. It is **not** a full agentic system.

The coordinator does not invent work; it walks a known sequence (rewrite → 8 scripts → slides → TTS → merge). `DirectPipelineRunner` is the more honest system: tools with an LLM bolted on. A full agentic system would **set or refine goals, perceive results, remember across runs, recover from partial failure, and fail a run that does not meet a machine check** — even if an agent said "Done."

If the product stays "one transcript in, one mp4 out," **direct mode + tools + checks** can be the complete system. Agents remain an optional control plane. A system that accepts mid-run goals such as "make it shorter and change the voice" is a different product. You prepare for that by keeping all side effects in `ProductionTools` and adding critic / resume / approve / verify — not by thickening coordinator prompts.

Harness vs agentic system

	Harness (see companion note)	Full agentic system
Question	Can the model only act through a controlled runtime?	Can the system pursue a goal, see the world, and correct itself?
Today	Partially present	Missing perception, resume, approve, verify, remember, recover
Stop condition	Coordinator prompt + failover if merge missing	Files + explicit success predicate

What is already present (confirmed)

- A **closed task**: transcript → 8 scripts → slides → audio → merged video.
- **Specialists** that call `ProductionTools` (they do not write PNGs, MP3s, or MP4s themselves).
- **Handoff / sequential** graphs plus **direct failover** if the merged file is missing.
- **Live pipeline status** injected into agent prompts (`ProductionContextProvider`).

- **Light guardrails** on agent chat only (not direct mode).
- **Per-run snapshots** under `output/agent-file-memory/production-notes-{RunId}/`.

What a full agentic system still needs

Layer	Needed	Today (confirmed)
Goal / contract	Explicit run goal + success predicate (files exist, counts, duration) that **fails the process** if unmet	Console `Done:` plus failover if merge missing; no post-run checker
Planner vs executor	A planner that decomposes *this* run (skip rewrite if stub, retry TTS 03 only)	Fixed specialist order; prompts hardcode "8"
Perception / critic	An agent or tool that **looks at outputs** — slide text vs script, audio duration vs 7 minutes, ffmpeg errors — and sends work back	No critic. Ollama writes slide sentences; nothing reviews the PNG or MP4
Human-in-the-loop	Pause after rewritten talk / slides before spending ElevenLabs	Fire-and-forget console run
Durable state / resume	Checkpoint: if `audio-06` fails, restart from 06	Wipe media on start; crash loses the run
Recovery	Retry, backoff, fallback (ElevenLabs → SAPI), budget cap	Throw and exit. If an ElevenLabs key is set, HTTP failure does not fall back to SAPI
Memory	Preferences and priors across runs (voice, brand, last good talk)	Per-run snapshots; not retrieval. See `docs/memory-vs-mem0.md`
Observability	Traces: tool, tokens, seconds, cost, which agent	Console logs
Eval loop	Golden fixture + quality gates; feed failures back into prompts/tools	No test project
Identity / tenancy	User/run isolation if it ever leaves one workstation	Named mutex, one process
Cancellation / budget	Stop Ollama/TTS on Ctrl+C; max spend / max seconds	`CancellationToken.None` on the expensive calls
Tool policy	Same validation on **every** path, including direct	Guardrails skip direct mode. See `docs/guardrails.md`

Optional later (only if the product becomes open-ended): dynamic tool discovery (MCP), RAG over extra source docs, parallel specialists, a scheduler. Those make a **platform**, not this factory.

What "fully agentic" should mean for this app

Not "more agents." A closed factory becomes agentic when:

1. **The world is the source of truth** — `pipeline-state.json` + files, not the model's last message.
2. **A critic can reject work** — e.g. "slide 03-02 is not one sentence" → re-run phrase rewrite only.
3. **Partial failure is recoverable** — resume; do not wipe everything unless the user asks.
4. **A human can approve the expensive step** — narration and slide text before TTS/video.
5. **A check fails the run** even if an agent said it finished.
6. **Traces exist** so you can change the harness from real failures.

Practical order (proposed)

Do these on the current architecture. Do not replace Microsoft Agent Framework first.

1. **Success predicate** — `HarnessChecks` after merge (8 scripts, ≤5 slides, 8 mp3s, merged file, status `complete`).
2. **Resume** — stop deleting good audio/video by default; wipe only with a flag.
3. **Cancellation + retries** — token through Ollama/TTS/ffmpeg; retry TTS per segment.
4. **Critic pass** — cheap first: duration vs `TargetTalkMinutes`, empty slides, missing files. Later: a model reviews slide sentences against the script.
5. **Approve gate** — write `narration-rewritten.txt`, print the path, wait for `--approve` or a keypress before TTS.
6. `events.jsonl` in `production-notes-{RunId}`.
7. **Option-driven prompts** — no hardcoded "8"; interpolate `StudioOptions`.
8. **Outer harness** — `AGENTS.md` + one test, so coding agents do not fight the factory.

Defer: mem0, extra specialists, MCP, multi-user, swapping MAF.

Target shape (proposed)

```
goal: produce video_{RunId}.mp4 from this transcript
    → plan (skip stages that already satisfy checks)
    → ProductionTools only
    → critic / HarnessChecks
    → if reject: retry the failing stage only
    → optional human approve before TTS
    → events.jsonl + pipeline-state.json + production-notes-{RunId}
    → fail the process if checks fail (even if the model said "Done")
```

Agents become interchangeable. Direct mode stays the reference executor.

Honest limit

If the product stays one talk → one video, do not build an open-ended planner. Build **perceive, resume, approve, verify, remember, recover** on top of `ProductionTools`. That is the agentic option for *this* application.

Key files (current system)

File	Role
`src/NarrationStudio/ProductionApp.cs`	Mode selection, failover to direct
`src/NarrationStudio/Orchestration/DirectPipelineRunner.cs`	Deterministic executor
`src/NarrationStudio/Tools/ProductionTools.cs`	Only I/O surface; keep it that way
`src/NarrationStudio/Agents/AgentFactory.cs`	Crew and prompts (hardcoded "8")
`src/NarrationStudio/Agents/WorkflowHost.cs`	Scripted graph, not a dynamic planner
`src/NarrationStudio/Services/PipelineStore.cs`	Run state; wipe-on-start; snapshots